

Smart Campus Facility Booking System

Course: ICS220 - 20606 Program. Fund.

Semester: Spring 2026

Assignment: Final Project

Professor:

Areej Abdulfattah

Team Members:

Sultan Alkatheeri - 202318578

Saif Alhashmi - 202323058

Zayed Alaghbari - 202380111

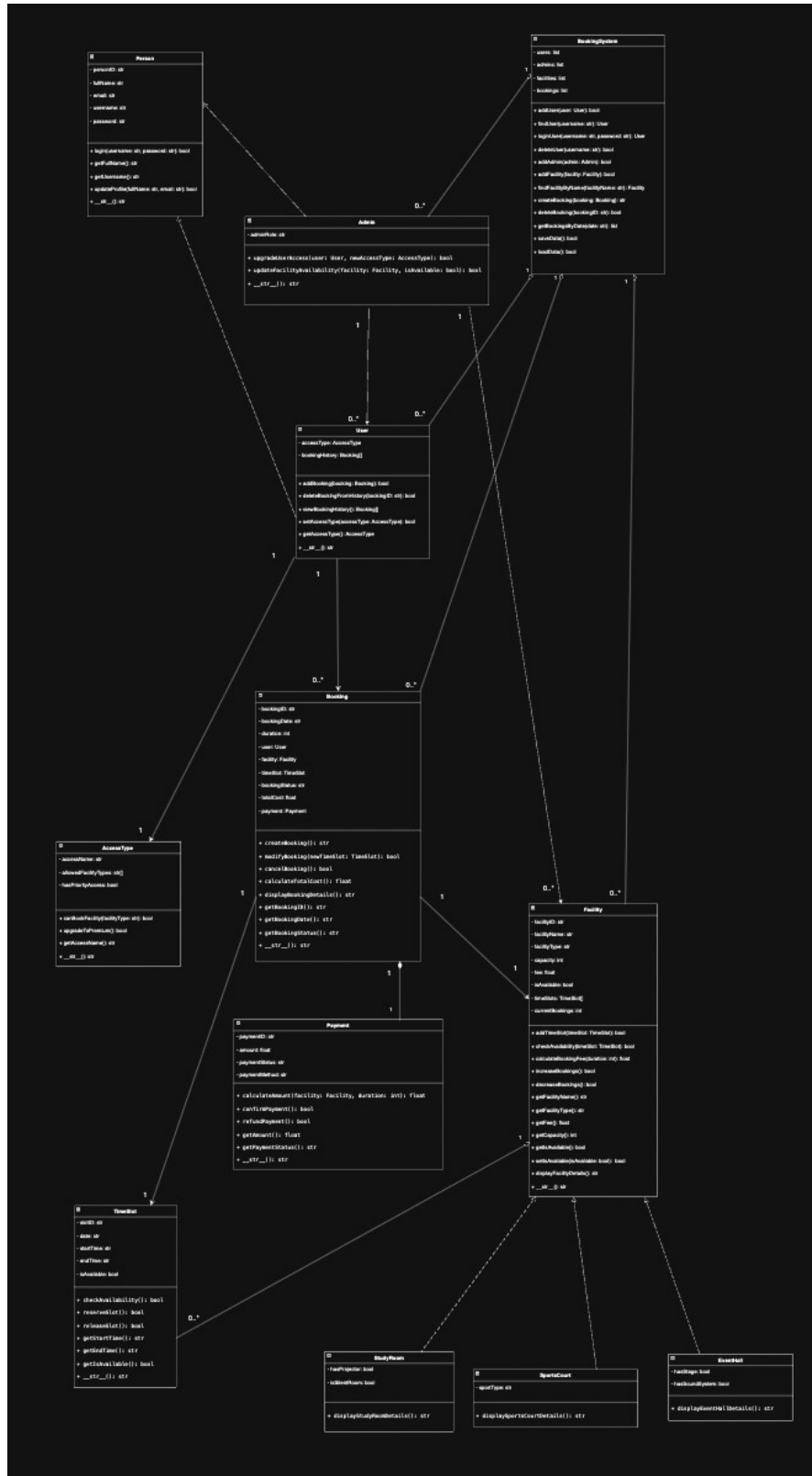
GitHub Link:

<https://github.com/Sultan-Alkatheeri/Smart-Campus-Booking-System.git>

UML Class Diagram Link:

https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=_blank&layers=1&nav=1&title=Team2%20Saif%20Sultan%20Zayed.drawio&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1w4FzVfsl7oNA2uJA4VCUL8gUggV9bwAL%26export%3Ddownload

UML Class Diagram



Description of UML Class Diagram

Access Specifier Definitions

Public (+): Public methods can be accessed from outside the class. In the diagram, the majority of methods are public since the GUI and other classes require access to them.

Private (-): The only way to access private attributes is within the same class. The attributes are private to protect the data and stop direct changes from outside of the class. So it supports encapsulation because the data is protected and accessed through methods.

Protected (#): Protected access is typically used with inheritance. In this diagram, there was no use of protected access since the classes have private attributes with public getter and setter methods.

Relationship Description

1. Person and User

Relationship: Inheritance

Description: User is a type of Person. the Person is the parent class and User is the child class which inherits from it so This is used because both Person and User share like the same basic account details beacues if this we do not need to repeat the same attributes again. This is inheritance because a user has the same basic account information as a person, like ID, name, email, username, and password, but like also has booking-related information, like access type and booking history.

Multiplicity: N/A

1. Person and Admin

Relationship: Inheritance

Description: Admin is a type of Person. the Person is the parent class and Admin is the child class that inherits from it. This is used because Admin also needs the same account details as a normal person, but it has extra admin responsibilities. This is inheritance since an admin still has the same account details as a person but adds admin actions like monitoring bookings, updating availability, and upgrading users.

Multiplicity: N/A

1. Facility and StudyRoom

Relationship: Inheritance

Description: StudyRoom is a type of Facility. Facility is the parent class and StudyRoom is the child class. This is used because StudyRoom has the same general facility information, but it also needs its own special details. This is inheritance since a study room has the same general facility details, like capacity, fee, availability, and time slots, but it also has study room features, like projector and silent room status.

Multiplicity: N/A

1. Facility and SportsCourt

Relationship: Inheritance

Description: SportsCourt is a type of Facility. Facility is the parent class and SportsCourt is the child class. This is used because SportsCourt still has normal facility details, but it needs extra sports details. This is inheritance because a sports court has the same common facility information as other facilities, but it has sport-specific information such as sport type and equipment requirement.

Multiplicity: N/A

1. Facility and EventHall

Relationship: Inheritance

Description: EventHall is a type of Facility. Facility is the parent class and EventHall is the child class. so it used because the EventHall is still a facility but like it has special event features that other facilities may not have so like it is inheritance because the event hall uses the same facility information but adds event-specific features such as stage and sound system.

Multiplicity: N/A

1. User and AccessType

Relationship: Binary Association

Description: A User is connected to an AccessType. so Its binary association because the user doesn't own the access type as a part object, but the user needs it to know what booking permissions are allowed. but like It also show the difference between Standard Access and Premium Access where like Premium can have priority access.

Multiplicity: One user has one access type (1..1).

1. User and Booking

Relationship: Binary Association Description: A User is connected to Booking because the users can like create, view, modify, and delete bookings. This is binary association because the User uses the Booking records, but the Booking is not a part object owned by the User. The Booking is connected to the user's activity and booking history. Multiplicity: so one user can have zero or many bookings (1 to 0..*).

1. Booking and Facility

Relationship: Binary Association Description: A Booking is connected to a Facility because like for every booking it must be like for one selected facility.so it is binary association because the Booking uses the Facility details, like capacity, availability, and fee, but it does not own the Facility and the Facility can still exist even if the Booking is deleted. Multiplicity: One facility can like have zero or many bookings but like each booking is connected to one facility (0..* to 1).

1. Booking and TimeSlot

Relationship: Binary Association Description: A Booking is connected to a TimeSlot because every booking need to use one selected time slot. This is binary association because the Booking

use TimeSlot to check and see time slot availability however the TimeSlot is not owned by the Booking so this also helps the system avoid double booking. Multiplicity: One booking uses one time slot (1..1).

1. Booking and Payment

Relationship: Composition Description: Payment is part of Booking so it is composition since like the payment record depend on the booking If the Booking is deleted the Payment should like also will deleted because the payment only there or it like exists for that booking and this shows a strong whole-part relationship. Multiplicity: One booking has one payment record (1..1).

1. Facility and TimeSlot

Relationship: Composition Description: TimeSlot is part of Facility so its composition because the like time slots belong to the facility schedule. If the Facility is removed from the system the TimeSlots should also will be removed because they need it and they are not useful without that facility so it show that TimeSlot depends on Facility. Multiplicity: One facility has one or more time slots (1 to 1..*).

1. BookingSystem and User

Relationship: Aggregation Description: BookingSystem has a collection of User objects. it is aggregation because the system manages users, but users are still separate objects with their own details and the methods The system stores the users, but they are not strong part objects like composition. Multiplicity: One booking system manages zero or many users (1 to 0..*).

1. BookingSystem and Admin

Relationship: Aggregation Description: BookingSystem has a collection of Admin objects. so this is aggregation since the admins is stored and managed by the system, however they are like still separate objects with their own details and methods. so the admins can exist as system users with admin responsibilities. Multiplicity: One booking system manages zero or many admins (1 to 0..*).

1. BookingSystem and Facility

Relationship: Aggregation Description: BookingSystem has a collection of Facility objects. its is aggregation because the system manages facilities, but like each facility is still a separate object with its own capacity, fee, availability, and time slots. so he system use the facilities for booking, but like it also does not make them a dependent part. Multiplicity: One booking system manages zero or many facilities (1 to 0..*).

1. BookingSystem and Booking

Relationship: Aggregation Description: BookingSystem has a collection of Booking objects. so its aggregation since the system store and also manages booking records, however for like each booking it still own object with details like date, facility, time slot and also total cost, and even payment record. so this also supports saving and loading booking data using Pickle. Multiplicity: One booking system manages zero or many bookings (1 to 0..*).

1. Admin and User

Relationship: Binary Association Description: Admin is connected to User because the admin can manage users and upgrade their access type. its binary association because the Admin interacts with the User for management tasks but like the Admin does not own the User For example, the admin can upgrade a user from Standard Access to Premium Access. Multiplicity: One admin can manage zero or many users (1 to 0..*).

1. Admin and Facility

Relationship: Binary Association Description: Admin is connected to Facility because the admin can like really update facility availability and monitor facility usage and capacity so this is binary association since the Admin interact with the Facility for management tasks but like the Admin doesn't own the Facility For example, the admin can make a facility unavailable during maintenance. Multiplicity: One admin can manage zero or many facilities (1 to 0..*).

1. Admin and Booking

Relationship: Binary Association Description: Admin is connected to Booking since the admin can view daily bookings and monitor booking activity. so its is binary association since the Admin use the Booking records to manage the system but like the Admin does not own the Booking For example, the admin can check how many bookings were made on one day. Multiplicity: One admin can view zero or many bookings (1 to 0..*).

Assumptions

1. Person is used as the parent class for User and Admin.
2. Facility is used as the parent class for StudyRoom, SportsCourt, and EventHall.
3. Each user has one access type at a time.
4. A new user can have zero bookings.
5. Each booking must be connected to one user, one facility, one time slot, and one payment record.
6. TimeSlot is a separate class because availability must be checked before booking.
7. Payment is included because some facilities require booking fees.
8. BookingSystem stores users, admins, facilities, and bookings as collections.
9. Pickle storage is handled through saveData() and loadData().

Python Code

```
# We import pickle because the assignment asks us to save objects in
binary files.
import pickle

# We import tkinter because the assignment asks for a GUI.
import tkinter as tk

# We import messagebox because it helps us show errors and
confirmations in the GUI.
from tkinter import messagebox
```

```

# Cell 2: AccessType, Person, User, Admin

# AccessType is used to control what each user is allowed to book.
class AccessType:
    """Class to represent Standard or Premium access."""

    # We store the access name and allowed facility types so the
    # system can check booking permission.
    def __init__(self, accessName, allowedFacilityTypes,
hasPriorityAccess):
        self.accessName = accessName
        self.allowedFacilityTypes = allowedFacilityTypes
        self.hasPriorityAccess = hasPriorityAccess

    # We use this method before booking so users cannot book outside
    # their access type.
    def canBookFacility(self, facilityType):
        if facilityType in self.allowedFacilityTypes:
            return True
        return False

    # This method upgrades the user to Premium and gives access to all
    # facility types.
    def upgradeToPremium(self):
        self.accessName = "Premium"
        self.allowedFacilityTypes = ["Study Room", "Sports Court",
"Event Hall"]
        self.hasPriorityAccess = True
        return True

    # This getter returns the access name.
    def getAccessName(self):
        return self.accessName

    # This method displays the access type as text.
    def __str__(self):
        return self.accessName

# Person is a parent class because User and Admin share the same basic
# account details.
class Person:
    """Class to represent a person in the system."""

    # We store common account details here to avoid repeating them in
    # User and Admin.
    def __init__(self, personID, fullName, email, username, password):
        self.personID = personID
        self.fullName = fullName
        self.email = email

```

```

        self.username = username
        self.password = password

    # This method checks if the entered username and password match
    # the saved account.
    def login(self, username, password):
        if self.username == username and self.password == password:
            return True
        return False

    # This getter returns the full name.
    def getFullName(self):
        return self.fullName

    # This getter returns the username.
    def getUsername(self):
        return self.username

    # This method updates profile details.
    def updateProfile(self, fullName, email):
        self.fullName = fullName
        self.email = email
        return True

    # This method displays the person as text.
    def __str__(self):
        return self.personID + " - " + self.fullName

# User inherits from Person because a user is a type of person.
class User(Person):
    """Class to represent a facility booking user."""

    # We use the parent constructor and add access type and booking
    # history.
    def __init__(self, personID, fullName, email, username, password,
        accessType):
        Person.__init__(self, personID, fullName, email, username,
            password)
        self.accessType = accessType
        self.bookingHistory = []

    # This method adds a booking to the user's history after a
    # successful booking.
    def addBooking(self, booking):
        self.bookingHistory.append(booking)
        return True

    # This method removes a booking from the user's history when it is
    # deleted.

```



```

def deleteBookingFromHistory(self, bookingID):
    for booking in self.bookingHistory:
        if booking.bookingID == bookingID:
            self.bookingHistory.remove(booking)
            return True
    return False

# This method returns all bookings made by the user.
def viewBookingHistory(self):
    return self.bookingHistory

# This method changes the user's access type.
def setAccessType(self, accessType):
    self.accessType = accessType
    return True

# This getter returns the user's access type.
def getAccessType(self):
    return self.accessType

# This method displays user details.
def __str__(self):
    return self.personID + " - " + self.fullName + " - " +
str(self.accessType)

# Admin inherits from Person because an admin is also a type of person.
class Admin(Person):
    """Class to represent an administrator."""

    # We use the parent constructor and add the admin role.
    def __init__(self, personID, fullName, email, username, password,
adminRole):
        Person.__init__(self, personID, fullName, email, username,
password)
        self.adminRole = adminRole

    # This method upgrades a user's access type.
    def upgradeUserAccess(self, user, newAccessType):
        user.setAccessType(newAccessType)
        return True

    # This method changes facility availability.
    def updateFacilityAvailability(self, facility, isAvailable):
        facility.setIsAvailable(isAvailable)
        return True

    # This method displays admin details.
    def __str__(self):

```

```

        return self.personID + " - " + self.fullName + " - " +
self.adminRole

# Cell 3: TimeSlot and Facility Classes

# TimeSlot is used because each facility can have different booking
times.
class TimeSlot:
    """Class to represent a booking time slot."""

    # We store the date and time details for each booking slot.
    def __init__(self, slotID, date, startTime, endTime, isAvailable):
        self.slotID = slotID
        self.date = date
        self.startTime = startTime
        self.endTime = endTime
        self.isAvailable = isAvailable

    # This method checks if the time slot is available.
    def checkAvailability(self):
        return self.isAvailable

    # This method reserves the slot after a successful booking.
    def reserveSlot(self):
        self.isAvailable = False
        return True

    # This method releases the slot if a booking is cancelled.
    def releaseSlot(self):
        self.isAvailable = True
        return True

    # This getter returns the start time.
    def getStartTime(self):
        return self.startTime

    # This getter returns the end time.
    def getEndTime(self):
        return self.endTime

    # This getter returns slot availability.
    def getIsAvailable(self):
        return self.isAvailable

    # This method displays the time slot as text.
    def __str__(self):
        return self.date + " " + self.startTime + " - " + self.endTime

# Facility is a parent class because StudyRoom, SportsCourt, and

```

EventHall share common details.

```
class Facility:
    """Class to represent a campus facility."""

    # We store common facility information here so child classes can
    # reuse it.
    def __init__(self, facilityID, facilityName, facilityType,
capacity, fee, isAvailable):
        self.facilityID = facilityID
        self.facilityName = facilityName
        self.facilityType = facilityType
        self.capacity = capacity
        self.fee = fee
        self.isAvailable = isAvailable
        self.timeSlots = []
        self.currentBookings = 0

    # This method adds a time slot to the facility.
    def addTimeSlot(self, timeSlot):
        self.timeSlots.append(timeSlot)
        return True

    # This method checks facility availability, time slot
    # availability, and capacity.
    def checkAvailability(self, timeSlot):
        if self.isAvailable == True and timeSlot.getIsAvailable() ==
True:
            if self.currentBookings < self.capacity:
                return True
            return False

    # This method calculates booking cost using fee and duration.
    def calculateBookingFee(self, duration):
        return self.fee * duration

    # This method increases the booking count after a successful
    # booking.
    def increaseBookings(self):
        self.currentBookings = self.currentBookings + 1
        return True

    # This method decreases booking count after cancellation.
    def decreaseBookings(self):
        if self.currentBookings > 0:
            self.currentBookings = self.currentBookings - 1
            return True
        return False

    # This getter returns the facility name.
    def getFacilityName(self):
```

```

        return self.facilityName

    # This getter returns the facility type.
    def getFacilityType(self):
        return self.facilityType

    # This getter returns the facility fee.
    def getFee(self):
        return self.fee

    # This getter returns the facility capacity.
    def getCapacity(self):
        return self.capacity

    # This getter returns facility availability.
    def getIsAvailable(self):
        return self.isAvailable

    # This setter changes facility availability.
    def setIsAvailable(self, isAvailable):
        self.isAvailable = isAvailable
        return True

    # This method displays facility details.
    def displayFacilityDetails(self):
        return (
            "Facility: " + self.facilityName +
            "\nType: " + self.facilityType +
            "\nCapacity: " + str(self.capacity) +
            "\nFee: " + str(self.fee) + " AED"
        )

    # This method displays the facility as text.
    def __str__(self):
        return self.facilityID + " - " + self.facilityName

# StudyRoom inherits from Facility because it is a type of facility.
class StudyRoom(Facility):
    """Class to represent a study room."""

    # We add study-room-specific details such as projector and silent
    # room.
    def __init__(self, facilityID, facilityName, capacity, fee,
isAvailable, hasProjector, isSilentRoom):
        Facility.__init__(self, facilityID, facilityName, "Study
Room", capacity, fee, isAvailable)
        self.hasProjector = hasProjector
        self.isSilentRoom = isSilentRoom

```

```

# This method displays study room details.
def displayStudyRoomDetails(self):
    return (
        self.displayFacilityDetails() +
        "\nProjector Available: " + str(self.hasProjector) +
        "\nSilent Room: " + str(self.isSilentRoom)
    )

# SportsCourt inherits from Facility because it is a type of facility.
class SportsCourt(Facility):
    """Class to represent a sports court."""

    # We add sports-court-specific details such as sport type.
    def __init__(self, facilityID, facilityName, capacity, fee,
isAvailable, sportType):
        Facility.__init__(self, facilityID, facilityName, "Sports
Court", capacity, fee, isAvailable)
        self.sportType = sportType

    # This method displays sports court details.
    def displaySportsCourtDetails(self):
        return (
            self.displayFacilityDetails() +
            "\nSport Type: " + self.sportType
        )

# EventHall inherits from Facility because it is a type of facility.
class EventHall(Facility):
    """Class to represent an event hall."""

    # We add event-hall-specific details such as stage and sound
system.
    def __init__(self, facilityID, facilityName, capacity, fee,
isAvailable, hasStage, hasSoundSystem):
        Facility.__init__(self, facilityID, facilityName, "Event
Hall", capacity, fee, isAvailable)
        self.hasStage = hasStage
        self.hasSoundSystem = hasSoundSystem

    # This method displays event hall details.
    def displayEventHallDetails(self):
        return (
            self.displayFacilityDetails() +
            "\nStage Available: " + str(self.hasStage) +
            "\nSound System Available: " + str(self.hasSoundSystem)
        )

# Cell 4: Payment and Booking Classes

```

```

# Payment is used because some facilities require payment after
booking.
class Payment:
    """Class to represent a payment."""

    # We store payment details so the system can track payment status
    and amount.
    def __init__(self, paymentID, amount, paymentStatus,
paymentMethod):
        self.paymentID = paymentID
        self.amount = amount
        self.paymentStatus = paymentStatus
        self.paymentMethod = paymentMethod

    # This method calculates payment amount based on facility fee and
    duration.
    def calculateAmount(self, facility, duration):
        self.amount = facility.calculateBookingFee(duration)
        return self.amount

    # This method changes payment status to Paid.
    def confirmPayment(self):
        self.paymentStatus = "Paid"
        return True

    # This method changes payment status back to Unpaid after
    cancellation.
    def refundPayment(self):
        self.paymentStatus = "Unpaid"
        return True

    # This getter returns the payment amount.
    def getAmount(self):
        return self.amount

    # This getter returns payment status.
    def getPaymentStatus(self):
        return self.paymentStatus

    # This method displays payment details as text.
    def __str__(self):
        return self.paymentID + " - " + str(self.amount) + " AED - " +
self.paymentStatus

# Booking is important because it connects the user, facility, time
slot, and payment together.
class Booking:
    """Class to represent a booking."""

```

```

        # We store all booking details here.
        def __init__(self, bookingID, bookingDate, duration, user,
facility, timeSlot):
            self.bookingID = bookingID
            self.bookingDate = bookingDate
            self.duration = duration
            self.user = user
            self.facility = facility
            self.timeSlot = timeSlot
            self.bookingStatus = "Confirmed"

        # We calculate total cost during booking creation.
        self.totalCost = facility.calculateBookingFee(duration)

        # We create a payment object connected to the booking.
        self.payment = Payment(
            "P" + bookingID,
            self.totalCost,
            "Unpaid",
            "Cash"
        )

        # This method checks all booking rules before confirming a
booking.
        def createBooking(self):

            # We first check if the user has permission to book this
facility type.
            if
self.user.getAccessType().canBookFacility(self.facility.getFacilityTyp
e()) == False:
                return "Access Denied"

            # We check if the facility itself is available.
            if self.facility.getIsAvailable() == False:
                return "Facility Not Available"

            # We check if the selected time slot is available.
            if self.timeSlot.getIsAvailable() == False:
                return "Time Slot Not Available"

            # We check capacity separately so the test output is clearer.
            if self.facility.currentBookings >=
self.facility.getCapacity():
                return "Capacity Full"

            # We reserve the time slot after successful booking.
            self.timeSlot.reserveSlot()

```

```

        # We increase the number of bookings for capacity tracking.
        self.facility.increaseBookings()

        # We add the booking to the user's booking history.
        self.user.addBooking(self)

        # We confirm payment automatically for this simple project
version.
        self.payment.confirmPayment()

        return "Booking Confirmed"

    # This method changes the booking time slot.
    def modifyBooking(self, newTimeSlot):

        # We only allow changing to an available slot.
        if newTimeSlot.getIsAvailable() == True:

            # We release the old slot first.
            self.timeSlot.releaseSlot()

            # We assign the new slot.
            self.timeSlot = newTimeSlot

            # We reserve the new slot.
            self.timeSlot.reserveSlot()

            return True

        return False

    # This method cancels the booking.
    def cancelBooking(self):

        # We change booking status to Cancelled.
        self.bookingStatus = "Cancelled"

        # We release the reserved time slot.
        self.timeSlot.releaseSlot()

        # We decrease booking count.
        self.facility.decreaseBookings()

        # We refund the payment.
        self.payment.refundPayment()

        return True

    # This method recalculates total cost if duration changes.
    def calculateTotalCost(self):
        self.totalCost =

```



```

self.facility.calculateBookingFee(self.duration)
    return self.totalCost

    # This method returns booking details in a clear format.
    def displayBookingDetails(self):
        return (
            "Booking ID: " + self.bookingID +
            "\nUser: " + self.user.getFullName() +
            "\nAccess Type: " +
self.user.getAccessType().getAccessName() +
            "\nFacility: " + self.facility.getFacilityName() +
            "\nFacility Type: " + self.facility.getFacilityType() +
            "\nDate: " + self.bookingDate +
            "\nTime Slot: " + self.timeSlot.getStartTime() + " - " +
self.timeSlot.getEndTime() +
            "\nDuration: " + str(self.duration) + " hour(s)" +
            "\nTotal Cost: " + str(self.totalCost) + " AED" +
            "\nPayment Status: " + self.payment.getPaymentStatus() +
            "\nBooking Status: " + self.bookingStatus
        )

    # This getter returns the booking ID.
    def getBookingID(self):
        return self.bookingID

    # This getter returns booking date.
    def getBookingDate(self):
        return self.bookingDate

    # This getter returns booking status.
    def getBookingStatus(self):
        return self.bookingStatus

    # This method displays booking as text.
    def __str__(self):
        return (
            self.bookingID +
            " - " +
            self.user.getFullName() +
            " - " +
            self.facility.getFacilityName()
        )

# Cell 5: BookingSystem Class

# BookingSystem is the main class that stores all users, admins,
# facilities, and bookings.
class BookingSystem:
    """Class to represent the smart campus booking system."""

```

```

    # We use lists because the system needs to store many users,
    facilities, and bookings.
    def __init__(self):
        self.users = []
        self.admins = []
        self.facilities = []
        self.bookings = []

    # This method adds a user only if the username is not already
    used.
    def addUser(self, user):
        if self.findUser(user.getUsername()) is None:
            self.users.append(user)
            return True
        return False

    # This method searches for a user by username.
    def findUser(self, username):
        for user in self.users:
            if user.getUsername() == username:
                return user
        return None

    # This method checks login details and returns the user if login
    is correct.
    def loginUser(self, username, password):
        user = self.findUser(username)
        if user is not None:
            if user.login(username, password) == True:
                return user
        return None

    # This method deletes a user from the system.
    def deleteUser(self, username):
        user = self.findUser(username)
        if user is not None:
            self.users.remove(user)
            return True
        return False

    # This method adds an admin to the system.
    def addAdmin(self, admin):
        self.admins.append(admin)
        return True

    # This method adds a facility to the system.
    def addFacility(self, facility):
        self.facilities.append(facility)
        return True

```

```

# This method searches for a facility by name.
def findFacilityByName(self, facilityName):
    for facility in self.facilities:
        if facility.getFacilityName() == facilityName:
            return facility
    return None

# This method creates a booking and stores it if the rules are correct.
def createBooking(self, booking):
    result = booking.createBooking()
    if result == "Booking Confirmed":
        self.bookings.append(booking)
    return result

# This method deletes a booking by booking ID.
def deleteBooking(self, bookingID):
    for booking in self.bookings:
        if booking.getBookingID() == bookingID:
            booking.cancelBooking()
            self.bookings.remove(booking)
            booking.user.deleteBookingFromHistory(bookingID)
            return True
    return False

# This method returns all bookings for one date.
def getBookingsByDate(self, date):
    dailyBookings = []
    for booking in self.bookings:
        if booking.getBookingDate() == date:
            dailyBookings.append(booking)
    return dailyBookings

# This method saves system data in a binary file using pickle.
def saveData(self):
    try:
        file = open("booking_system.dat", "wb")
        pickle.dump(self.users, file)
        pickle.dump(self.admins, file)
        pickle.dump(self.facilities, file)
        pickle.dump(self.bookings, file)
        file.close()
        return True
    except FileNotFoundError:
        return False

# This method loads system data from the binary pickle file.
def loadData(self):
    try:
        file = open("booking_system.dat", "rb")

```

```

        self.users = pickle.load(file)
        self.admins = pickle.load(file)
        self.facilities = pickle.load(file)
        self.bookings = pickle.load(file)
        file.close()
        return True
    except FileNotFoundError:
        return False
    except EOFError:
        return False

# Cell 6: Sample Data and Testing

# We create Standard access because Standard users can only book study
rooms.
standardAccess = AccessType("Standard", ["Study Room"], False)

# We create Premium access because Premium users can book all facility
types.
premiumAccess = AccessType("Premium", ["Study Room", "Sports Court",
"Event Hall"], True)

# We create the main system object to store all data.
system = BookingSystem()

# We create one user with Standard access to test booking
restrictions.
user1 = User("U001", "Sultan Saif Zayed Team 2", "team2@zu.ac.ae",
"team2", "1234", standardAccess)

# We create one admin to test admin-related features later.
admin1 = Admin("A001", "Admin User", "admin@zu.ac.ae", "admin",
"admin123", "Facility Manager")

# We add the user to the system.
system.addUser(user1)

# We add the admin to the system.
system.addAdmin(admin1)

# We create time slots for the facilities.
slot1 = TimeSlot("T001", "2026-05-08", "9:00", "11:00", True)
slot2 = TimeSlot("T002", "2026-05-08", "4:00", "6:00", True)
slot3 = TimeSlot("T003", "2026-05-08", "1:00", "3:00", True)

# We create facilities for the system.
studyRoom1 = StudyRoom("F001", "Study Room A", 6, 0.0, True, True,
True)
basketballCourt = SportsCourt("F002", "Basketball Court", 10, 20.0,
True, "Basketball")

```

```
eventHall1 = EventHall("F003", "Hall 1", 50, 100.0, True, True, True)

# We connect each time slot to its facility.
studyRoom1.addTimeSlot(slot1)
basketballCourt.addTimeSlot(slot2)
eventHall1.addTimeSlot(slot3)

# We add the facilities to the system.
system.addFacility(studyRoom1)
system.addFacility(basketballCourt)
system.addFacility(eventHall1)

# We test login using the correct username and password.
loggedUser = system.loginUser("team2", "1234")

# We print the login result.
print("Login successful:", loggedUser is not None)

# We create a study room booking because Standard users are allowed to
book study rooms.
booking1 = Booking("B001", "2026-05-08", 2, user1, studyRoom1, slot1)

# We try to create the study room booking.
result1 = system.createBooking(booking1)

# We print the study room booking result.
print("Study room booking result:", result1)

# We print the booking details if the booking was confirmed.
if result1 == "Booking Confirmed":
    print(booking1.displayBookingDetails())

# We create a basketball court booking to test Standard access
restriction.
booking2 = Booking("B002", "2026-05-08", 2, user1, basketballCourt,
slot2)

# We try to book the basketball court as a Standard user.
result2 = system.createBooking(booking2)

# We print the basketball booking result.
print("Basketball booking result:", result2)

# We upgrade the user to Premium to test the access upgrade
requirement.
user1.setAccessType(premiumAccess)

# We create another basketball court booking after the user becomes
Premium.
booking3 = Booking("B003", "2026-05-08", 2, user1, basketballCourt,
```

```

slot2)

# We try the basketball booking again after upgrade.
result3 = system.createBooking(booking3)

# We print the basketball booking result after upgrade.
print("Basketball booking after upgrade:", result3)

# We print booking details if the booking was confirmed.
if result3 == "Booking Confirmed":
    print(booking3.displayBookingDetails())

# We save the system data using pickle.
saveResult = system.saveData()

# We print the save result.
print("Data saved:", saveResult)

# We create a new system object to test loading from the pickle file.
newSystem = BookingSystem()

# We load the saved data.
loadResult = newSystem.loadData()

# We print the load result.
print("Data loaded:", loadResult)

# We print the number of loaded users.
print("Users loaded:", len(newSystem.users))

# We print the number of loaded bookings.
print("Bookings loaded:", len(newSystem.bookings))

# Cell 7: Complete Test Cases

print("\n--- Extra Test Cases ---")

# Test 1: Login failure
wrongLogin = system.loginUser("team2", "wrongpass")
print("Login failure test:", wrongLogin is None)

# Test 2: Create a new user account
user2 = User("U002", "New Student", "newstudent@zu.ac.ae", "student2",
"2222", standardAccess)
addUserResult = system.addUser(user2)
print("Create user account test:", addUserResult)

# Test 3: Display user details
print("Display user details:", user2)

# Test 4: Modify user profile

```

```

user2.updateProfile("Updated Student", "updatedstudent@zu.ac.ae")
print("Modify user profile test:", user2.getFullName())

# Test 5: Delete user account
deleteUserResult = system.deleteUser("student2")
print("Delete user test:", deleteUserResult)

# We change the user back to Standard to properly test access
restriction.
user1.setAccessType(standardAccess)

# Test 6: Event hall booking with Standard access should be denied
booking4 = Booking("B004", "2026-05-08", 2, user1, eventHall1, slot3)
result4 = system.createBooking(booking4)
print("Standard user event hall booking test:", result4)

# Test 7: Admin upgrades user access
adminUpgradeResult = admin1.upgradeUserAccess(user1, premiumAccess)
print("Admin upgrade user access test:", adminUpgradeResult)
print("User access after admin upgrade:",
user1.getAccessType().getAccessName())

# Test 8: Event hall booking after Premium upgrade
booking5 = Booking("B005", "2026-05-08", 2, user1, eventHall1, slot3)
result5 = system.createBooking(booking5)
print("Premium user event hall booking test:", result5)

# Test 9: Daily booking activity
bookingsToday = system.getBookingsByDate("2026-05-08")
print("Daily booking activity count:", len(bookingsToday))

# Test 10: Admin changes facility availability
admin1.updateFacilityAvailability(eventHall1, False)
print("Admin changed event hall availability:",
eventHall1.getIsAvailable())

# Test 11: Booking unavailable facility should fail
slot4 = TimeSlot("T004", "2026-05-08", "6:00", "8:00", True)
eventHall1.addTimeSlot(slot4)
booking6 = Booking("B006", "2026-05-08", 2, user1, eventHall1, slot4)
result6 = system.createBooking(booking6)
print("Unavailable facility booking test:", result6)

# We make the event hall available again for later tests.
admin1.updateFacilityAvailability(eventHall1, True)

# Test 12: Unavailable time slot should fail
booking7 = Booking("B007", "2026-05-08", 2, user1, basketballCourt,
slot2)
result7 = system.createBooking(booking7)

```

```

print("Unavailable time slot booking test:", result7)

# Test 13: Modify booking time slot
slot5 = TimeSlot("T005", "2026-05-08", "7:00", "9:00", True)
basketballCourt.addTimeSlot(slot5)
modifyResult = booking3.modifyBooking(slot5)
print("Modify booking test:", modifyResult)

# Test 14: Delete booking
deleteBookingResult = system.deleteBooking("B003")
print("Delete booking test:", deleteBookingResult)

# Test 15: View booking history
print("User booking history count:", len(user1.viewBookingHistory()))

# Test 16: Capacity full test
smallRoom = StudyRoom("F004", "Small Study Room", 1, 0.0, True, False,
True)
smallSlot1 = TimeSlot("T006", "2026-05-09", "10:00", "11:00", True)
smallSlot2 = TimeSlot("T007", "2026-05-09", "11:00", "12:00", True)
smallRoom.addTimeSlot(smallSlot1)
smallRoom.addTimeSlot(smallSlot2)
system.addFacility(smallRoom)

booking8 = Booking("B008", "2026-05-09", 1, user1, smallRoom,
smallSlot1)
result8 = system.createBooking(booking8)
print("Small room first booking:", result8)

booking9 = Booking("B009", "2026-05-09", 1, user1, smallRoom,
smallSlot2)
result9 = system.createBooking(booking9)
print("Capacity full test:", result9)

# Test 17: Save data again after all changes
saveResult2 = system.saveData()
print("Save after test cases:", saveResult2)

# Test 18: Load data again after all changes
loadedSystem = BookingSystem()
loadResult2 = loadedSystem.loadData()
print("Load after test cases:", loadResult2)
print("Loaded users:", len(loadedSystem.users))
print("Loaded facilities:", len(loadedSystem.facilities))
print("Loaded bookings:", len(loadedSystem.bookings))

# Cell 9: Admin Dashboard GUI

# This function opens the admin dashboard window.
def openAdminDashboard():

```



```

adminWindow = tk.Toplevel()
adminWindow.title("Admin Dashboard")
adminWindow.geometry("450x500")

titleLabel = tk.Label(adminWindow, text="Admin Dashboard")
titleLabel.pack(pady=10)

# This label shows daily booking activity.
bookingsToday = system.getBookingsByDate("2026-05-08")
bookingText = "Bookings Today: " + str(len(bookingsToday))

bookingLabel = tk.Label(adminWindow, text=bookingText)
bookingLabel.pack(pady=5)

# This part shows all facility usage and capacity.
usageText = ""
for facility in system.facilities:
    usageText = usageText + facility.getFacilityName()
    usageText = usageText + " Capacity: " +
str(facility.currentBookings)
    usageText = usageText + "/" + str(facility.getCapacity()) + "\n"

usageLabel = tk.Label(adminWindow, text=usageText)
usageLabel.pack(pady=10)

# This function makes Hall 1 unavailable.
def makeHallUnavailable():
    admin1.updateFacilityAvailability(eventHall1, False)
    messagebox.showinfo("Admin Action", "Hall 1 is now
unavailable.")

# This function upgrades the current user to Premium.
def upgradeCurrentUser():
    admin1.upgradeUserAccess(currentUser, premiumAccess)
    messagebox.showinfo("Admin Action", "User upgraded to
Premium.")

unavailableButton = tk.Button(
    adminWindow,
    text="Make Hall 1 Unavailable",
    command=makeHallUnavailable
)
unavailableButton.pack(pady=5)

upgradeButton = tk.Button(
    adminWindow,
    text="Upgrade User to Premium",
    command=upgradeCurrentUser
)

```

```

        upgradeButton.pack(pady=5)

# Cell 8: Login GUI, Booking Interface, and User Actions

# This function shows facility details when the user selects a
facility.
def showFacilityDetails():
    facilityName = selectedFacility.get()
    facility = system.findFacilityByName(facilityName)
    if facility is not None:
        details = facility.displayFacilityDetails()
        facilityDetailsLabel.config(text=details)

# This function confirms a booking from the GUI.
def confirmBooking():
    facilityName = selectedFacility.get()
    facility = system.findFacilityByName(facilityName)
    if facilityName == "Study Room A":
        selectedSlot = slot1
    elif facilityName == "Basketball Court":
        selectedSlot = slot2
    else:
        selectedSlot = slot3
    booking = Booking("B" + str(len(system.bookings) + 1), "2026-05-
08", 2, currentUser, facility, selectedSlot)
    result = system.createBooking(booking)
    if result == "Booking Confirmed":
        messagebox.showinfo("Booking Successful",
booking.displayBookingDetails())
    else:
        messagebox.showerror("Booking Failed", result)

# This function displays the user's booking history.
def showBookingHistory():
    history = currentUser.viewBookingHistory()
    if len(history) == 0:
        messagebox.showinfo("Booking History", "No bookings found.")
    else:
        historyText = ""
        for booking in history:
            historyText = historyText + str(booking) + "\n"
        messagebox.showinfo("Booking History", historyText)

# This function deletes the last booking for a simple GUI test.
def deleteLastBooking():
    history = currentUser.viewBookingHistory()
    if len(history) == 0:
        messagebox.showerror("Delete Booking", "No booking to
delete.")
    else:

```

```

        lastBooking = history[-1]
        result = system.deleteBooking(lastBooking.getBookingID())
        if result == True:
            messagebox.showinfo("Delete Booking", "Booking deleted
successfully.")
        else:
            messagebox.showerror("Delete Booking", "Booking was not
deleted.")

# This function updates the user's profile with simple test values.
def updateProfile():
    currentUser.updateProfile("Sultan Saif Zayed", "sultan@zu.ac.ae")
    messagebox.showinfo("Profile Updated", "Profile updated
successfully.")

# This function shows the user details.
def showUserDetails():
    messagebox.showinfo("User Details", str(currentUser))

# This function saves data using pickle.
def saveSystemData():
    result = system.saveData()
    if result == True:
        messagebox.showinfo("Save Data", "Data saved successfully.")
    else:
        messagebox.showerror("Save Data", "Data was not saved.")

# This function opens the booking window after login.
def openBookingWindow(user):
    global selectedFacility
    global facilityDetailsLabel
    global currentUser
    currentUser = user
    bookingWindow = tk.Toplevel()
    bookingWindow.title("Smart Campus Booking System")
    bookingWindow.geometry("450x600")
    titleLabel = tk.Label(bookingWindow, text="Welcome " +
user.getFullName() + "\nAccess Type: " +
user.getAccessType().getAccessName())
    titleLabel.pack(pady=10)
    selectedFacility = tk.StringVar()
    selectedFacility.set("Study Room A")
    facilityMenu = tk.OptionMenu(bookingWindow, selectedFacility,
"Study Room A", "Basketball Court", "Hall 1")
    facilityMenu.pack(pady=5)
    detailsButton = tk.Button(bookingWindow, text="Show Facility
Details", command=showFacilityDetails)
    detailsButton.pack(pady=5)
    facilityDetailsLabel = tk.Label(bookingWindow, text="")
    facilityDetailsLabel.pack(pady=10)

```

```

        bookingButton = tk.Button(bookingWindow, text="Confirm Booking",
command=confirmBooking)
        bookingButton.pack(pady=5)
        historyButton = tk.Button(bookingWindow, text="View Booking
History", command=showBookingHistory)
        historyButton.pack(pady=5)
        deleteButton = tk.Button(bookingWindow, text="Delete Last
Booking", command=deleteLastBooking)
        deleteButton.pack(pady=5)
        profileButton = tk.Button(bookingWindow, text="Update Profile",
command=updateProfile)
        profileButton.pack(pady=5)
        userDetailsButton = tk.Button(bookingWindow, text="Display User
Details", command=showUserDetails)
        userDetailsButton.pack(pady=5)
        saveButton = tk.Button(
            bookingWindow,
            text="Save Data",
            command=saveSystemData
        )
        saveButton.pack(pady=5)

        adminButton = tk.Button(
            bookingWindow,
            text="Open Admin Dashboard",
            command=openAdminDashboard
        )
        adminButton.pack(pady=5)

        showFacilityDetails()

# This function checks login details.
def loginButtonClicked():
    username = usernameEntry.get()
    password = passwordEntry.get()
    user = system.loginUser(username, password)
    if user is not None:
        messagebox.showinfo("Login Successful", "Welcome " +
user.getFullName())
        openBookingWindow(user)
    else:
        messagebox.showerror("Login Failed", "Incorrect username or
password")

# This is the main login window.
loginWindow = tk.Tk()
loginWindow.title("Smart Campus Login")
loginWindow.geometry("350x250")
loginTitle = tk.Label(loginWindow, text="Smart Campus Facility
Booking")

```

```
loginTitle.pack(pady=10)
usernameLabel = tk.Label(loginWindow, text="Username")
usernameLabel.pack()
usernameEntry = tk.Entry(loginWindow)
usernameEntry.pack(pady=5)
passwordLabel = tk.Label(loginWindow, text="Password")
passwordLabel.pack()
passwordEntry = tk.Entry(loginWindow, show="*")
passwordEntry.pack(pady=5)
loginButton = tk.Button(loginWindow, text="Login",
command=loginButtonClicked)
loginButton.pack(pady=10)
```

```
loginWindow.mainloop()
```

Login successful: True

Study room booking result: Booking Confirmed

Booking ID: B001

User: Sultan Saif Zayed Team 2

Access Type: Standard

Facility: Study Room A

Facility Type: Study Room

Date: 2026-05-08

Time Slot: 9:00 - 11:00

Duration: 2 hour(s)

Total Cost: 0.0 AED

Payment Status: Paid

Booking Status: Confirmed

Basketball booking result: Access Denied

Basketball booking after upgrade: Booking Confirmed

Booking ID: B003

User: Sultan Saif Zayed Team 2

Access Type: Premium

Facility: Basketball Court

Facility Type: Sports Court

Date: 2026-05-08

Time Slot: 4:00 - 6:00

Duration: 2 hour(s)

Total Cost: 40.0 AED

Payment Status: Paid

Booking Status: Confirmed

Data saved: True

Data loaded: True

Users loaded: 1

Bookings loaded: 2

--- Extra Test Cases ---

Login failure test: True

Create user account test: True

Display user details: U002 - New Student - Standard

```
Modify user profile test: Updated Student
Delete user test: True
Standard user event hall booking test: Access Denied
Admin upgrade user access test: True
User access after admin upgrade: Premium
Premium user event hall booking test: Booking Confirmed
Daily booking activity count: 3
Admin changed event hall availability: False
Unavailable facility booking test: Facility Not Available
Unavailable time slot booking test: Time Slot Not Available
Modify booking test: True
Delete booking test: True
User booking history count: 2
Small room first booking: Booking Confirmed
Capacity full test: Capacity Full
Save after test cases: True
Load after test cases: True
Loaded users: 1
Loaded facilities: 4
Loaded bookings: 3
```

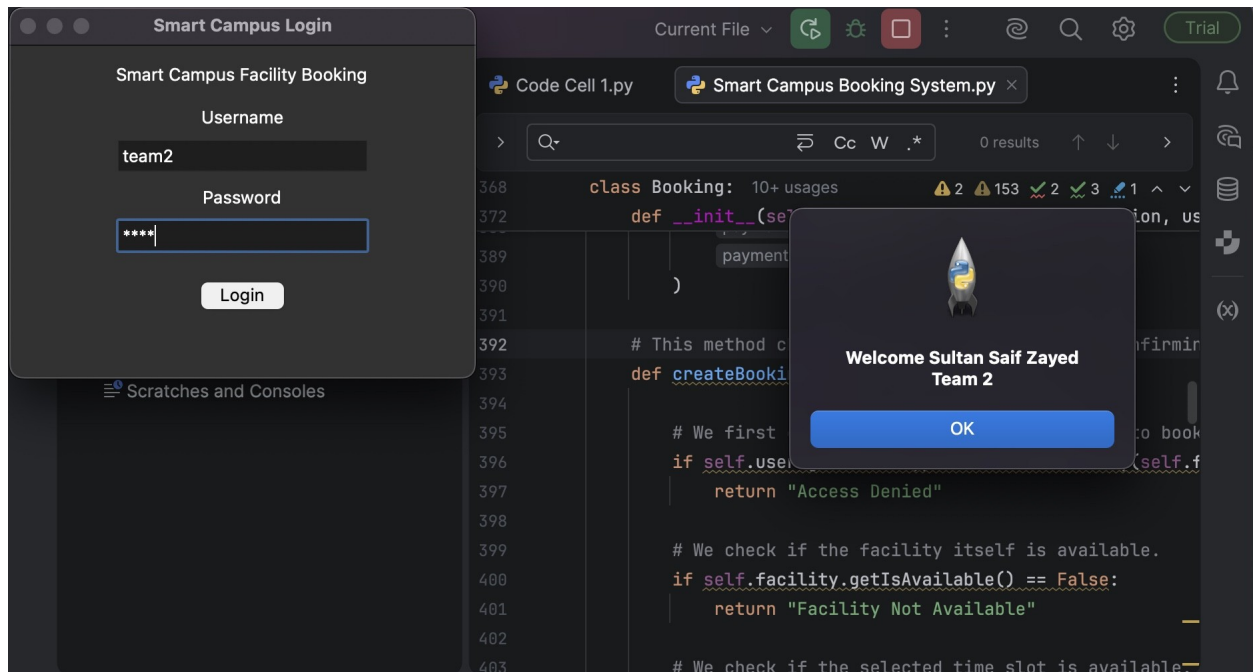
```
-----
-----
TclError                                Traceback (most recent call
last)
/tmp/ipykernel_19914/3615582817.py in <cell line: 0>()
    1006
    1007 # This is the main login window.
-> 1008 loginWindow = tk.Tk()
    1009 loginWindow.title("Smart Campus Login")
    1010 loginWindow.geometry("350x250")

/usr/lib/python3.12/tkinter/__init__.py in __init__(self, screenName,
baseName, className, useTk, sync, use)
    2347         baseName = baseName + ext
    2348         interactive = False
-> 2349         self.tk = _tkinter.create(screenName, baseName,
className, interactive, wantobjects, useTk, sync, use)
    2350         if _debug:
    2351             self.tk.settrace(_print_command)

TclError: no display name and no $DISPLAY environment variable
```

Login Success

This screenshot shows a successful login using the correct username and password credentials.



Main User Dashboard

This screenshot shows the main Smart Campus Booking System dashboard with facility options and booking management functions available to the user.



Smart Campus Booking System

Welcome Sultan Saif Zayed Team 2
Access Type: Premium

Study Room A 

Show Facility Details

Facility: Study Room A
Type: Study Room
Capacity: 6
Fee: 0.0 AED

Confirm Booking

View Booking History

Delete Last Booking

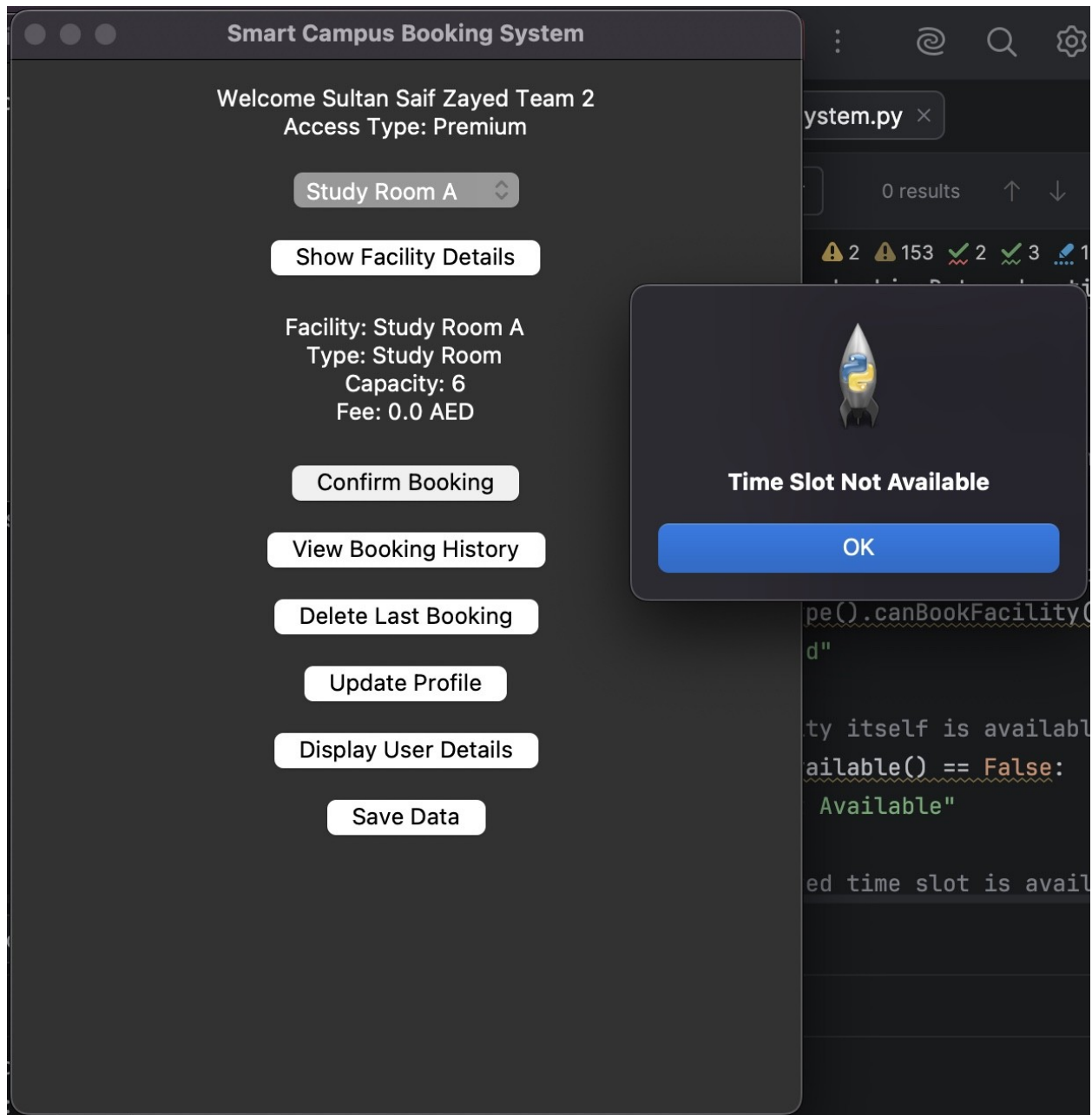
Update Profile

Display User Details

Save Data

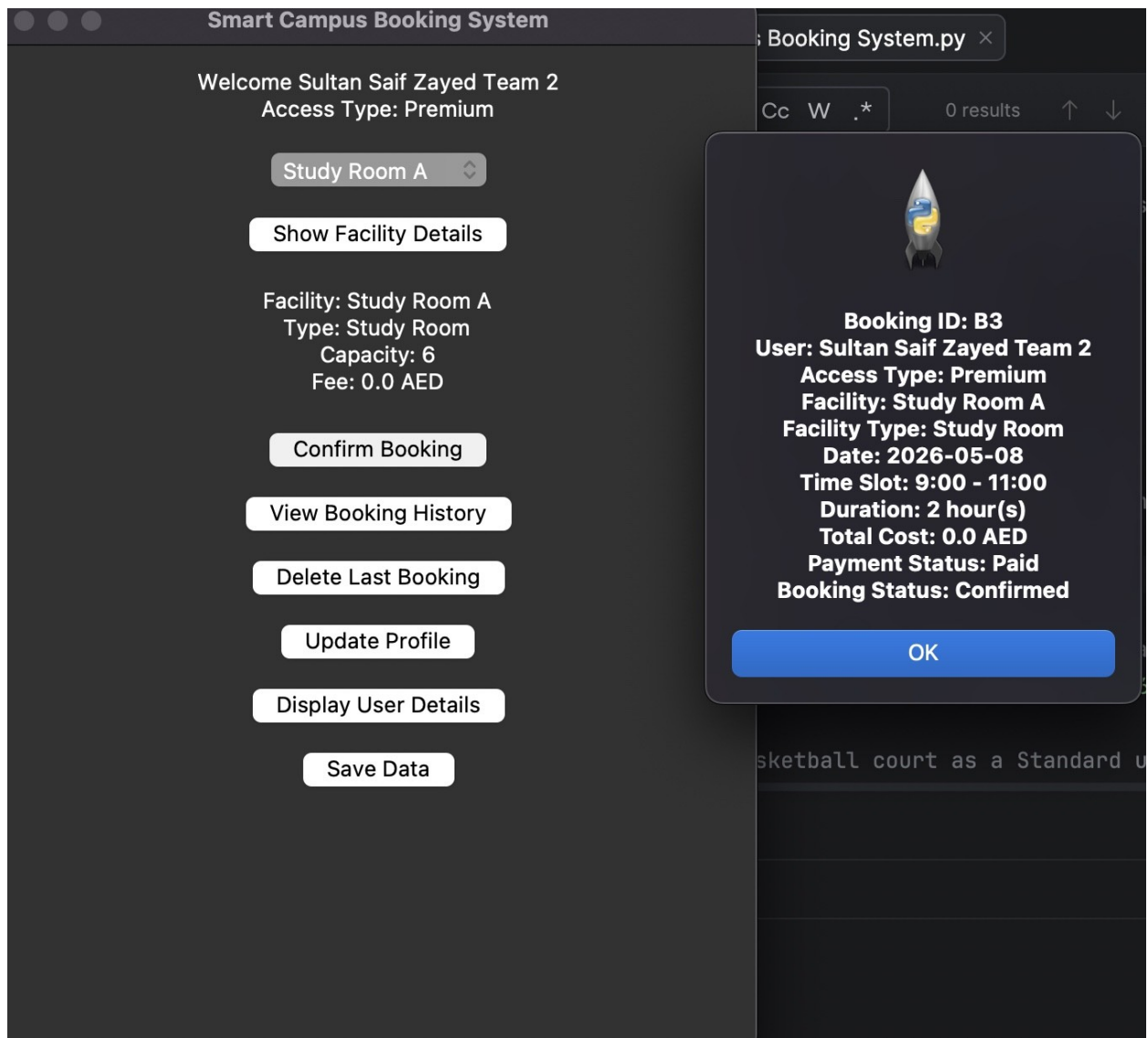
Time Slot Not Available

This screenshot shows the system validation message when a selected booking time slot is already unavailable.



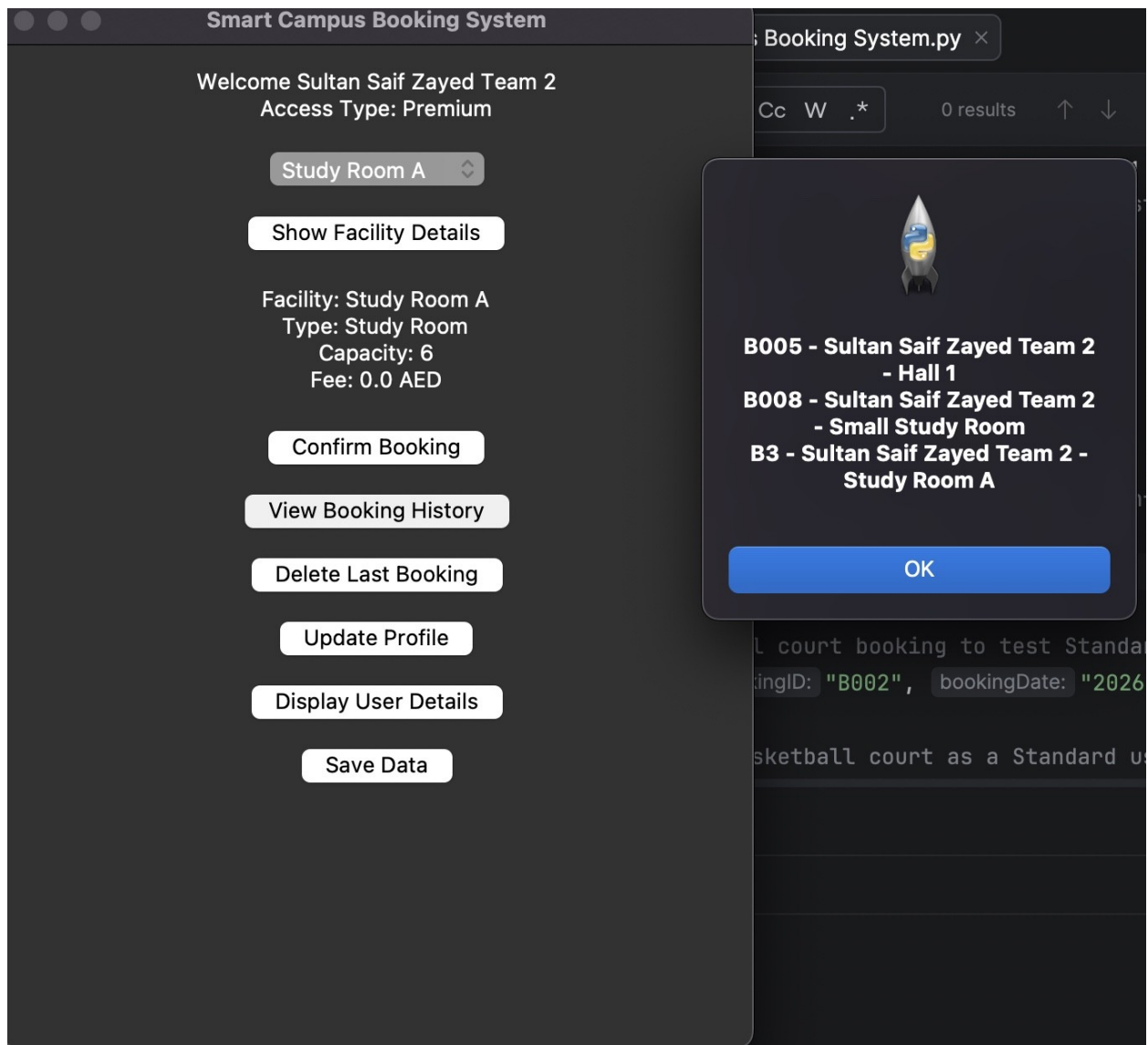
Successful Booking Confirmation

This screenshot shows a successful booking confirmation including booking ID, facility details, booking status, payment status, date, duration, and total cost.



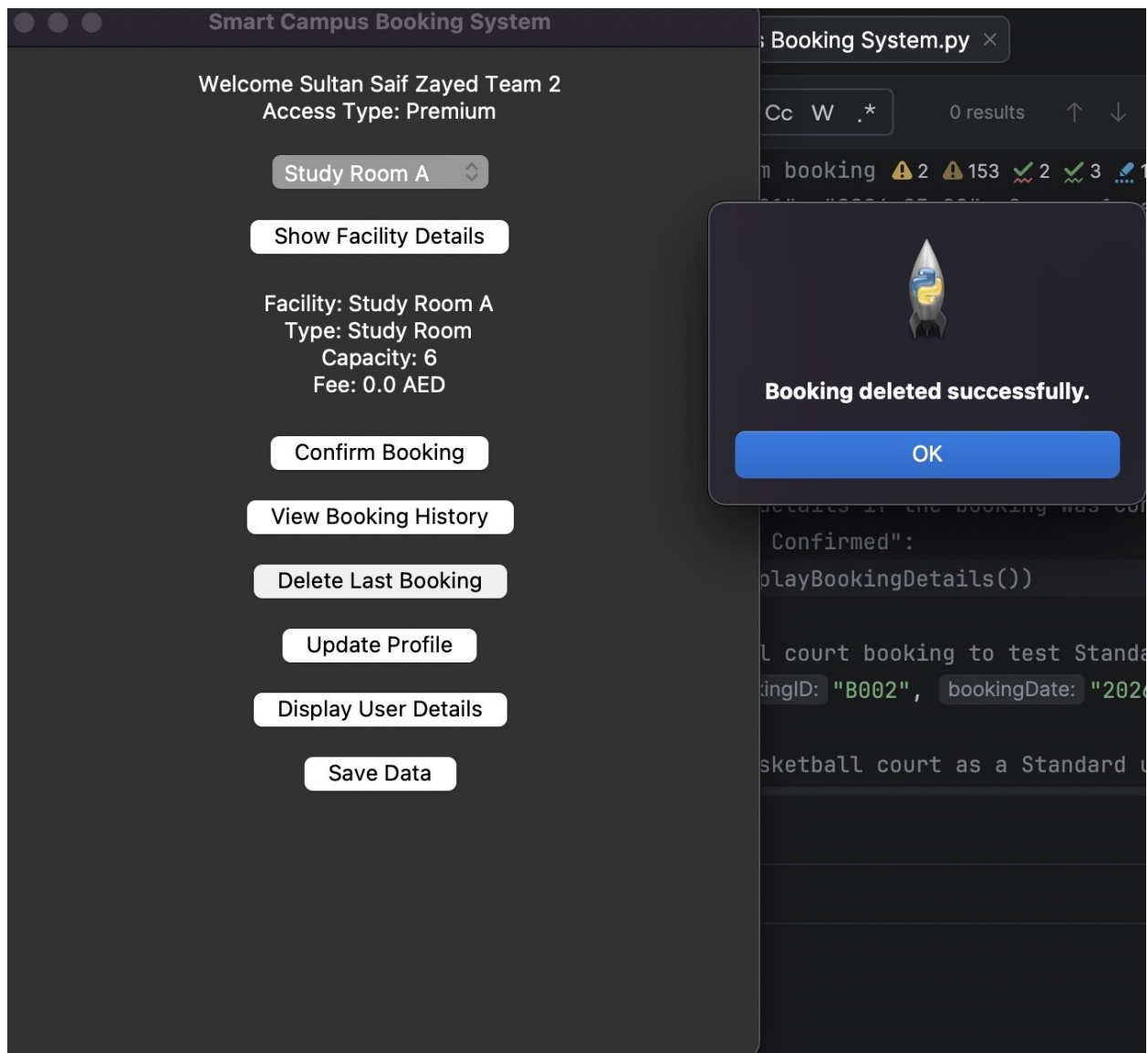
Booking History

This screenshot shows the booking history feature displaying all previous bookings made by the user.



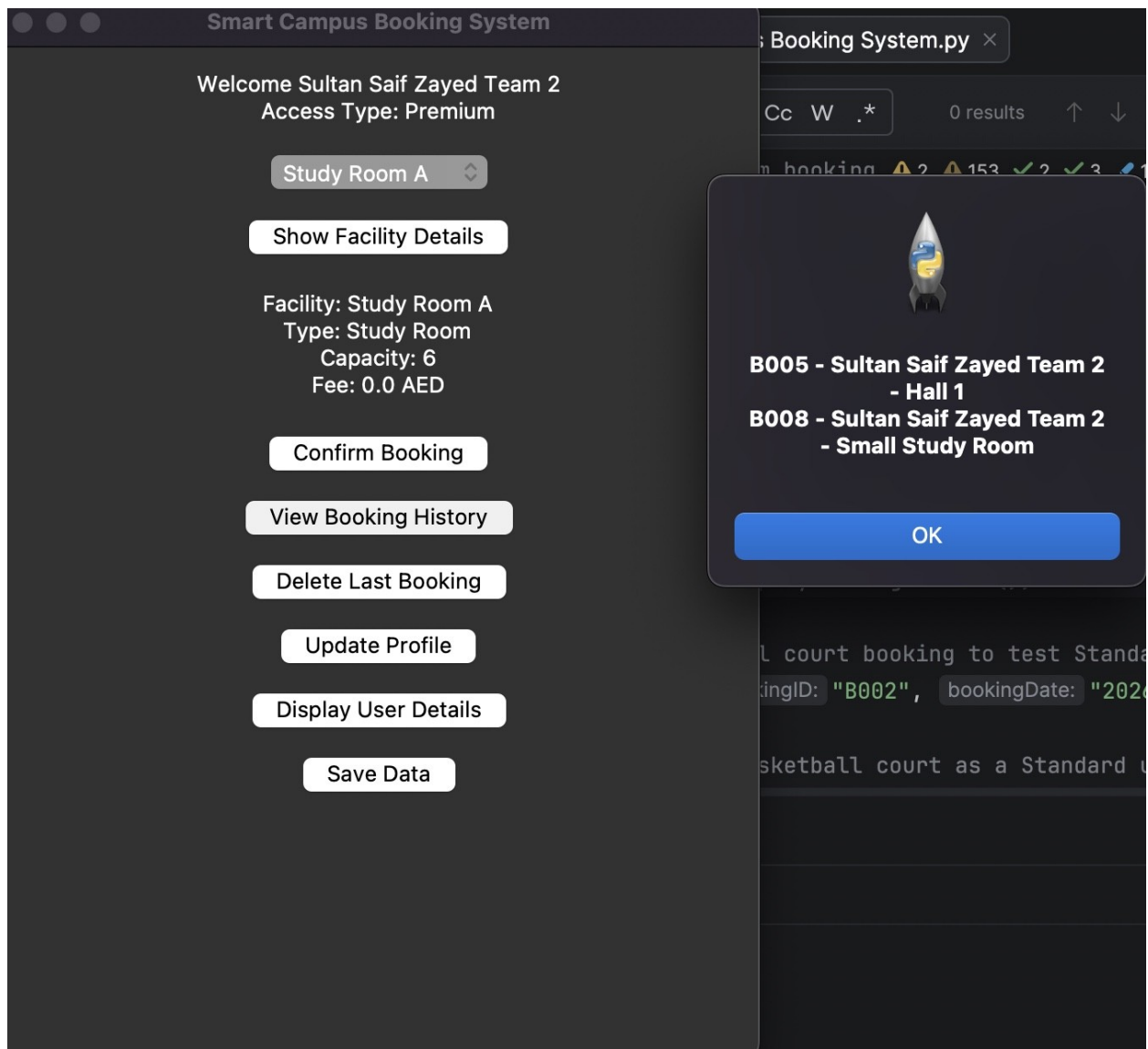
Booking Deleted Successfully

This screenshot shows the successful deletion of the user's most recent booking from the booking history.



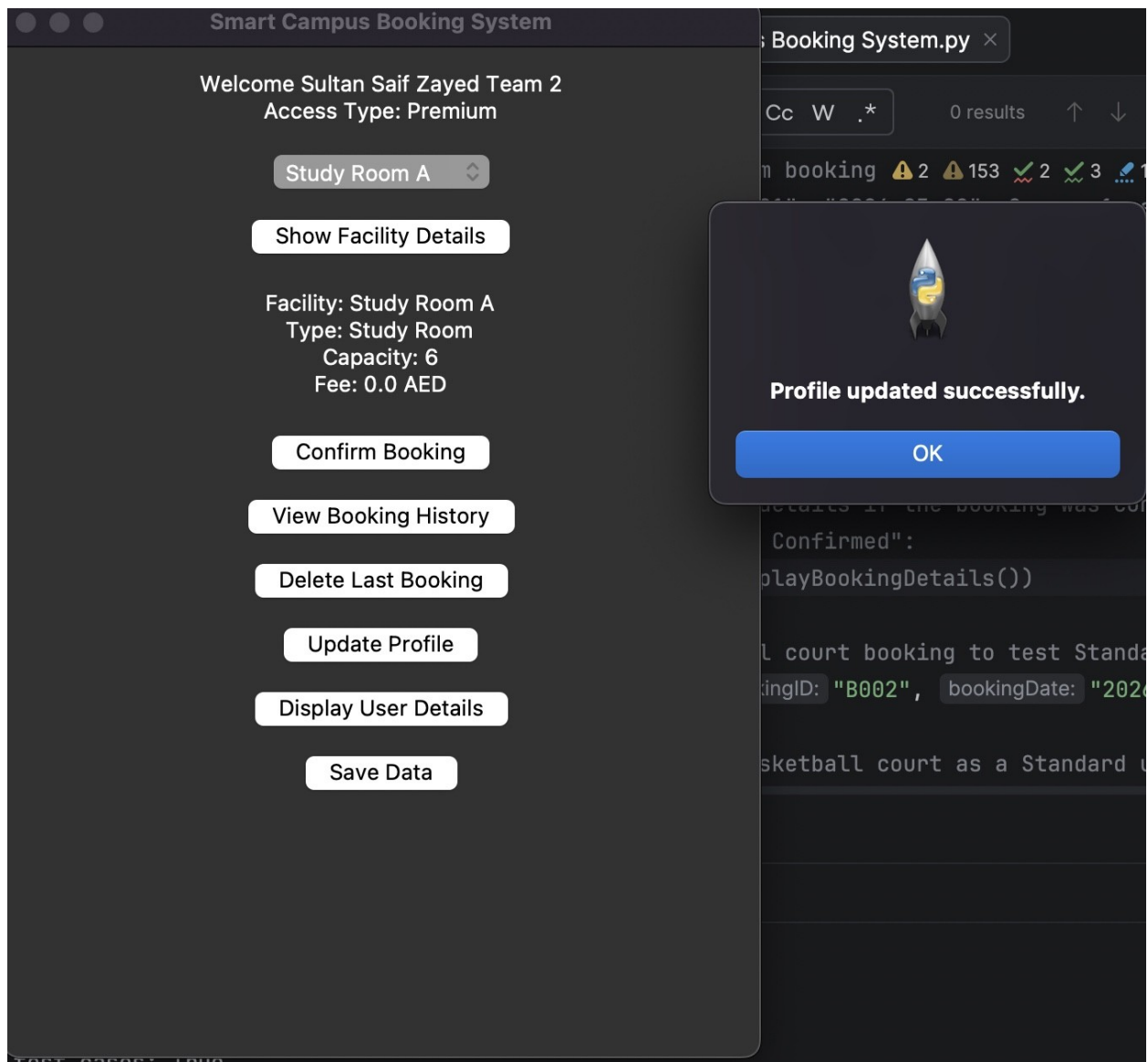
Booking History

This screenshot shows the booking history feature displaying previous bookings made by the user, including booking IDs and selected facilities.



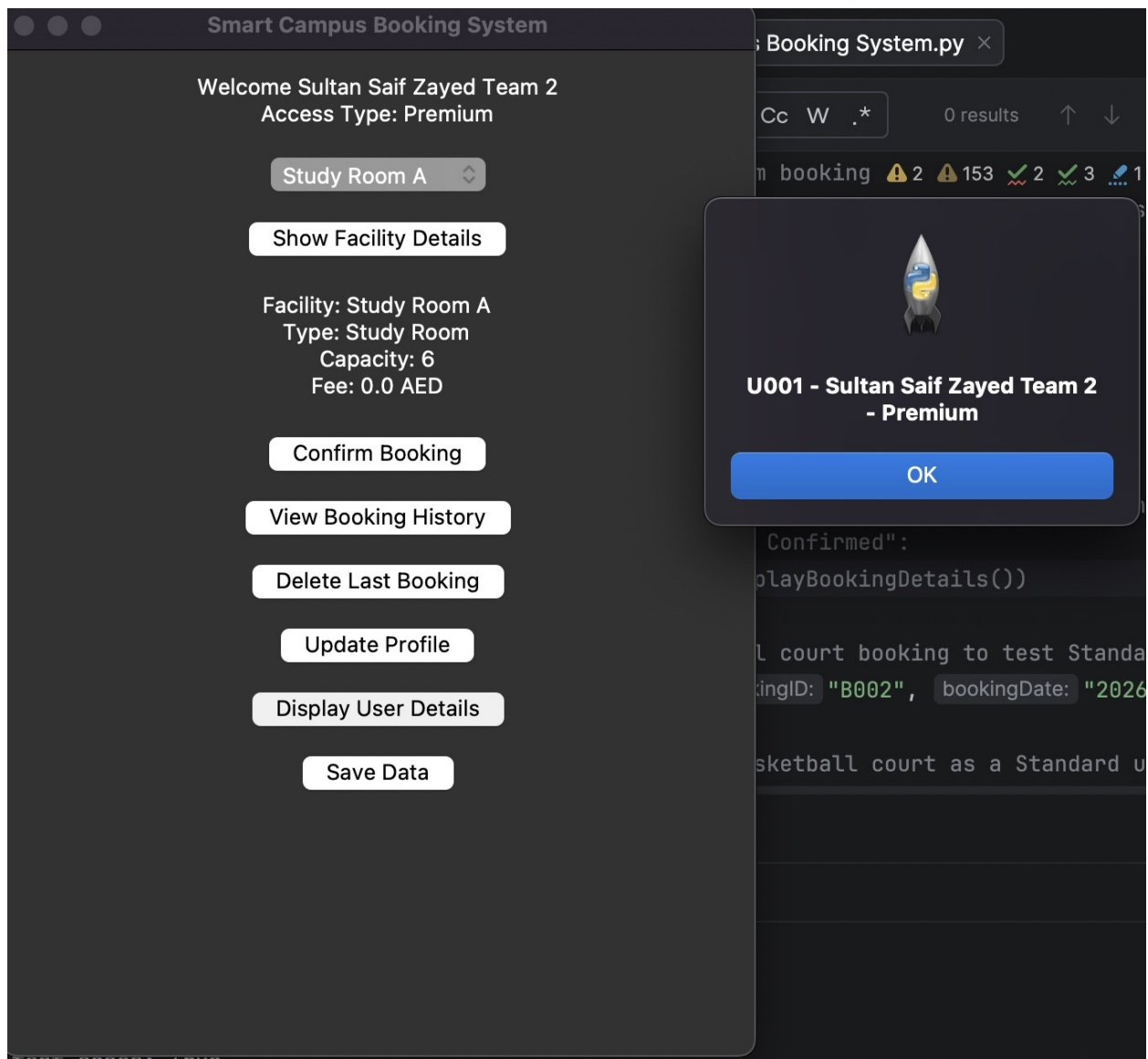
Updated Booking History

This screenshot shows the booking history after deleting a booking successfully.



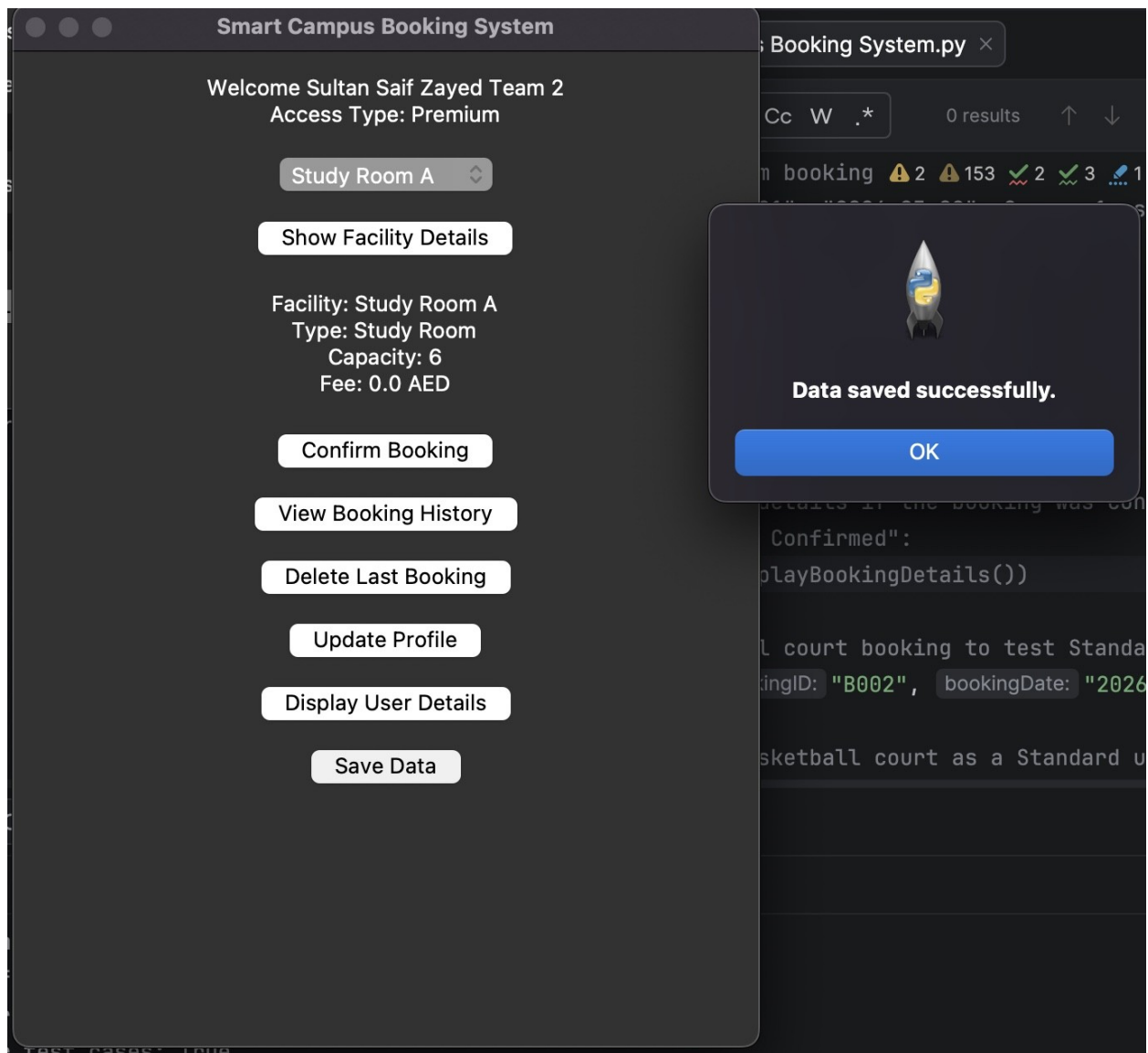
Updated User Details

This screenshot shows the updated user information after modifying the user profile successfully.



Data Saved Successfully

This screenshot shows the system successfully saving booking and user data into the data file.



Console Output and Test Results

This screenshot shows the console output after running the Smart Campus Booking System. It demonstrates successful login validation, booking confirmations, access control testing, payment processing, and data persistence using Pickle storage. The output also confirms that booking information, users, and saved data were loaded successfully after testing.

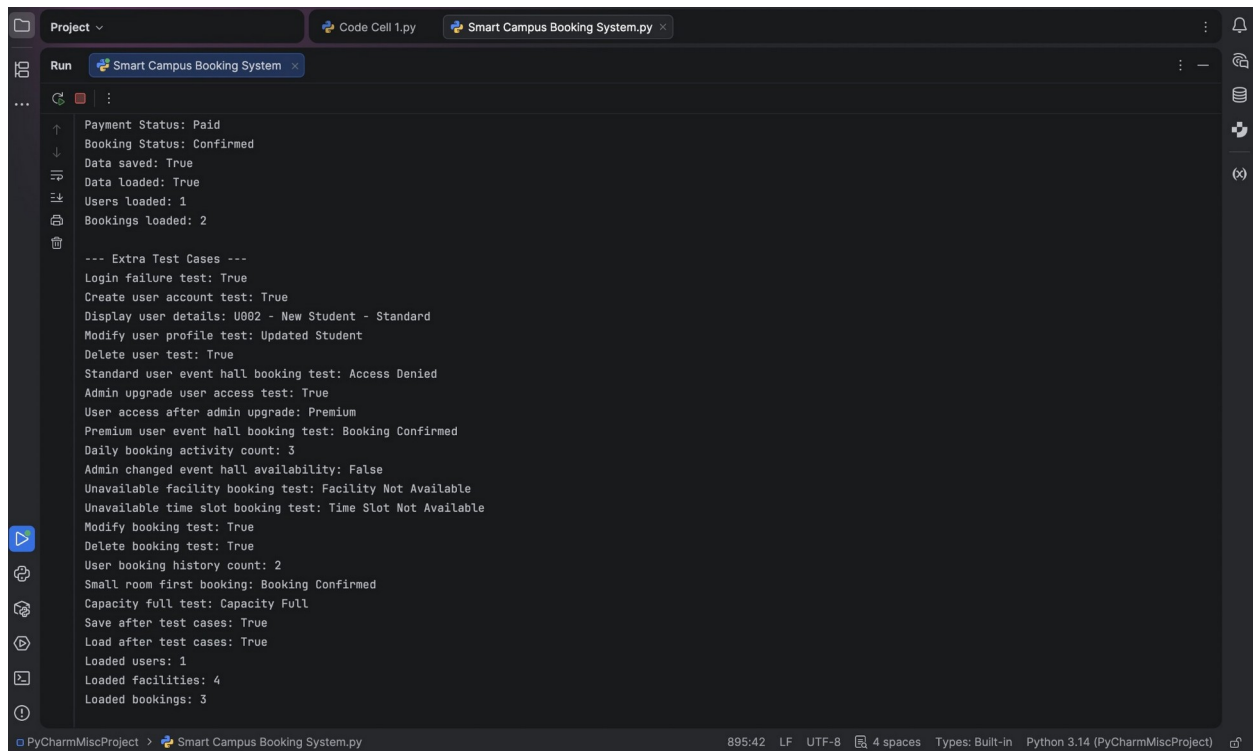

```

/Users/University/PyCharmMiscProject/.venv/bin/python /Users/University/PyCharmMiscProject/Smart Campus Booking System.py
Login successful: True
Study room booking result: Booking Confirmed
Booking ID: B001
User: Sultan Saif Zayed Team 2
Access Type: Standard
Facility: Study Room A
Facility Type: Study Room
Date: 2026-05-08
Time Slot: 9:00 - 11:00
Duration: 2 hour(s)
Total Cost: 0.0 AED
Payment Status: Paid
Booking Status: Confirmed
Basketball booking result: Access Denied
Basketball booking after upgrade: Booking Confirmed
Booking ID: B003
User: Sultan Saif Zayed Team 2
Access Type: Premium
Facility: Basketball Court
Facility Type: Sports Court
Date: 2026-05-08
Time Slot: 4:00 - 6:00
Duration: 2 hour(s)
Total Cost: 40.0 AED
Payment Status: Paid
Booking Status: Confirmed
Data saved: True
Data loaded: True
Users loaded: 1
Bookings loaded: 2

```

Additional Test Cases

This screenshot shows additional system test cases used to verify different functionalities of the Smart Campus Booking System. The tests include login failure validation, user account creation, profile updates, access upgrades by the admin, facility availability management, booking modification and deletion, capacity limit testing, booking history checks, and Pickle save/load functionality.

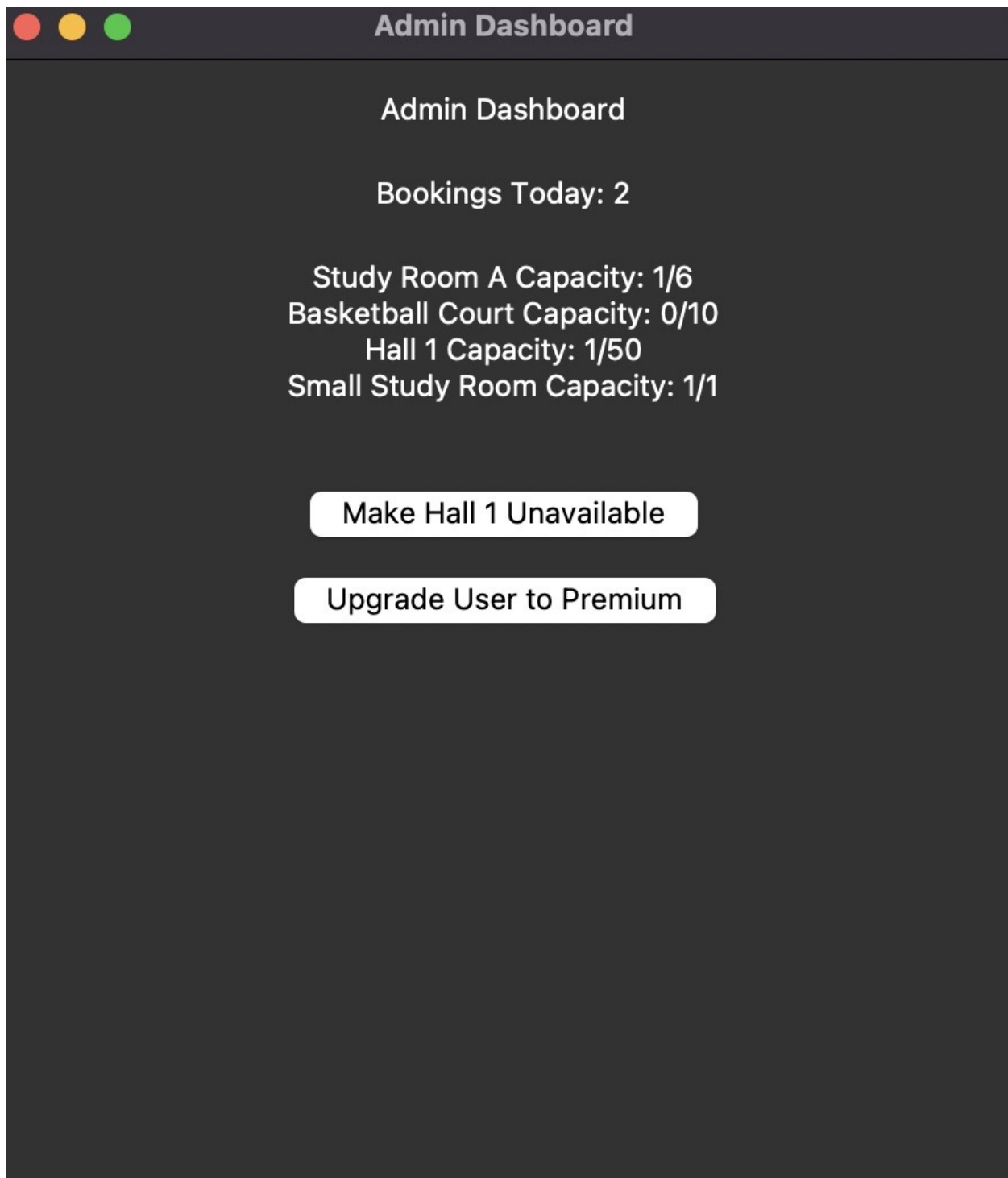


```
Payment Status: Paid
Booking Status: Confirmed
Data saved: True
Data loaded: True
Users loaded: 1
Bookings loaded: 2

--- Extra Test Cases ---
Login failure test: True
Create user account test: True
Display user details: U002 - New Student - Standard
Modify user profile test: Updated Student
Delete user test: True
Standard user event hall booking test: Access Denied
Admin upgrade user access test: True
User access after admin upgrade: Premium
Premium user event hall booking test: Booking Confirmed
Daily booking activity count: 3
Admin changed event hall availability: False
Unavailable facility booking test: Facility Not Available
Unavailable time slot booking test: Time Slot Not Available
Modify booking test: True
Delete booking test: True
User booking history count: 2
Small room first booking: Booking Confirmed
Capacity full test: Capacity Full
Save after test cases: True
Load after test cases: True
Loaded users: 1
Loaded facilities: 4
Loaded bookings: 3
```

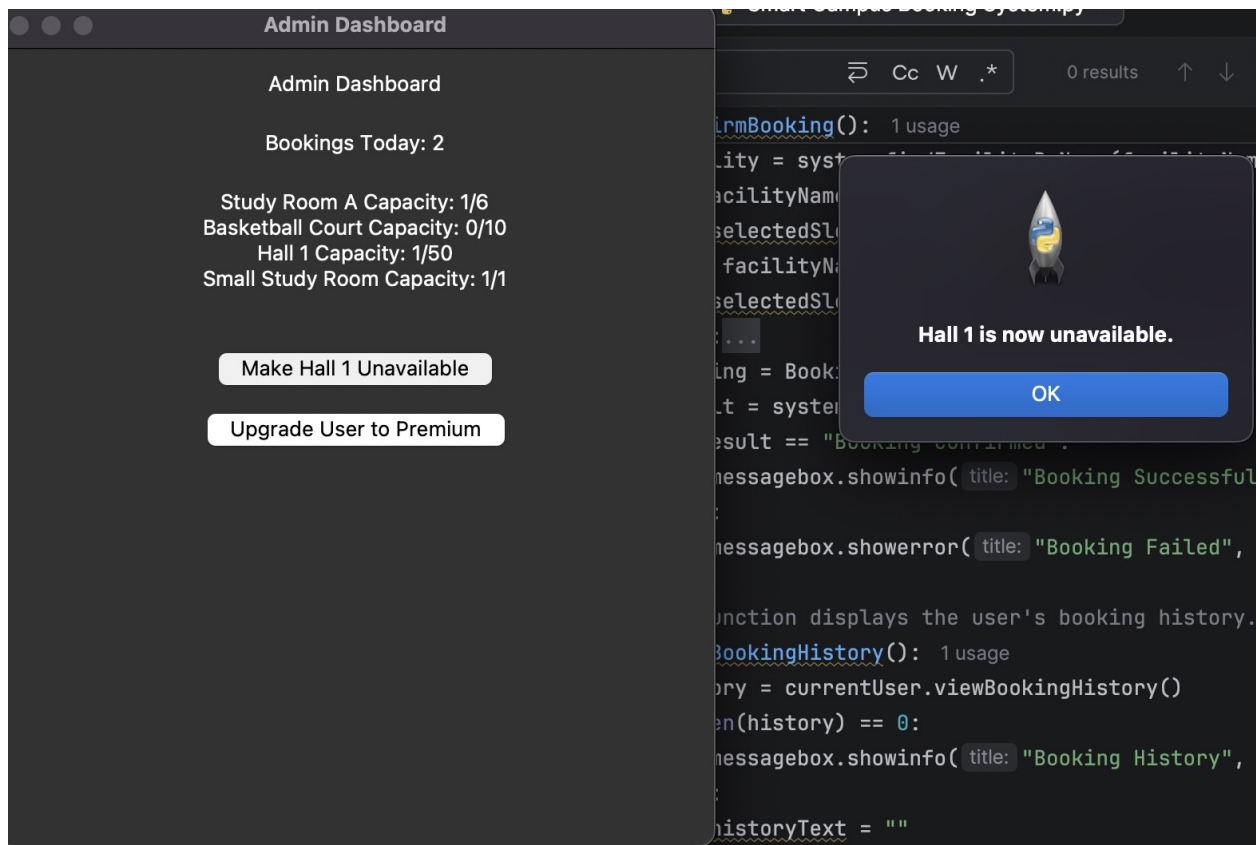
Admin Dashboard

This screenshot shows the Admin Dashboard interface of the Smart Campus Booking System. The dashboard allows the admin to monitor daily booking activity, view facility capacities, manage facility availability, and upgrade users from Standard access to Premium access.



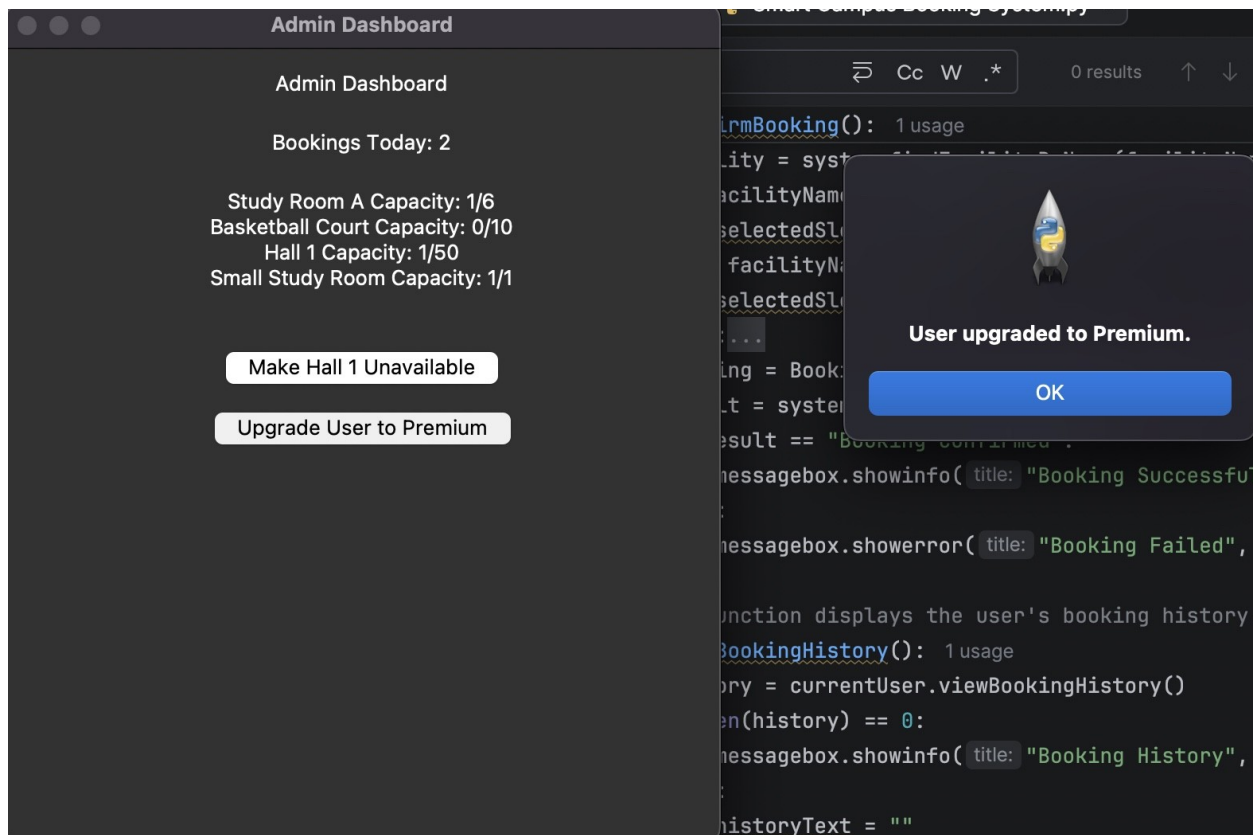
Facility Availability Management

This screenshot shows the admin using the facility management feature to make Hall 1 unavailable. The confirmation message demonstrates that the system successfully updates facility availability status to prevent future bookings during maintenance or restricted periods.



User Access Upgrade

This screenshot shows the admin upgrading a user from Standard access to Premium access. The system confirms the upgrade successfully, allowing the user to access premium facilities and additional booking permissions.



Summary of Learnings

In this project, we got an idea of how object-oriented programming concepts are used in a real system with Python. Inheritance, encapsulation, aggregation, composition and association are some of the key concepts used in OOP that we have used in designing the Smart Campus Booking System. Before implementation, we also learned about the modeling of relationship between classes using UML class diagram.

Moreover, we were able to develop a graphical user interface (GUI) using Tkinter and integrate it with the backend logic. The project also enabled us to learn bookings and user management, access permissions, payments and everything within a full software system.

We have learned how to test the functionalities of the system with different test cases such as booking validation, capacity checking, access control, profile update, admin operations etc. We also learned how to use Pickle storage to save and load system data to persist it.

Lastly, the project helped us develop our problem solving, debugging, team working, and system designing skills, and demonstrated the application of software engineering concepts to build a working and useful real-life application.